

CS496: Undergraduate Project I - Final Report

Analyzing Secure Sorting Algorithms and Extending the PRAC Framework for Graph Algorithms

Hitarth Makawana
Roll No: 230479
hitarthkm23@iitk.ac.in

Ronav Puri
Roll No: 230815
ronavgp23@iitk.ac.in

April 24, 2026

1 Introduction & Background

Multi-Party Computation (MPC) is a sub-domain of cryptography that allows multiple parties to jointly compute a function over their private inputs without revealing sensitive information. The foundational principle of MPC relies on two aspects: correctness and privacy, which together establish the notion of security.

Information can be distributed using various forms of secret sharing, including Additive, XOR, and Binary Secret Sharing. For each party, the received share appears as a seemingly random value, but these shares can be combined by the parties to facilitate joint computations. This project explores the domain of secure multi-party sorting and random-access data structures, culminating in the development of a novel Oblivious Breadth-First Search (OBFS) algorithm.

Traditional sorting algorithms fundamentally rely on data-dependent branching during comparison, which explicitly reveals the relative order of items and memory access patterns. Therefore, secure MPC sorting must be data-oblivious—capable of successfully sorting without leaking underlying values.

2 Research Summary: Secure Multi-Party Sorting and Applications

Kristján Valur Jónsson, Gunnar Kreitz, Misbah Uddin (2011)

2.1 Premise

The paper proposes a solution for secure data sorting and aggregation among n mutually distrustful parties $(P_1, \dots, P_n)$. The core setting involves parties who wish to compute a joint function on their private datasets without revealing individual inputs.

- **MP Sorting:** Parties $P_1, \dots, P_n$ hold secret sequences of elements $\mathbf{a}^{(1)}, \dots, \mathbf{a}^{(n)}$, respectively. The goal is to produce a sorted union of these sequences $\mathbf{a}$ without revealing elements of each party to any other party.
- **Weighted Set Intersection:** Inputs are (key,

weight) pairs from all parties. The aim is to sum the weights for all inputs with identical keys, and output unique keys with aggregate weights.

- **Top- k & IDS:** The framework is applied to a *Co-operative Intrusion Detection System*. Organizations secretly share logs of (IP address, traffic) to detect distributed attacks (e.g., slow port scans) by identifying heavy hitters (top- k anomalous IPs) without exposing sensitive local traffic data.

2.2 Sorting Algorithm

Algorithm 1 ODD-EVEN MERGE

Input: Sequence $\mathbf{a}$ whose two halves $a_1, a_2, \dots, a_{n/2}$ and $a_{n/2+1}, a_{n/2+2}, \dots, a_n$ are sorted. Length n is a power of 2.

Output: Sequence $\mathbf{a}$ is modified in-place to be sorted.

```

if  $n > 2$  then
    ODD-EVEN MERGE( $a_1, a_3, a_5, \dots, a_{n-1}$ )
    ODD-EVEN MERGE( $a_2, a_4, a_6, \dots, a_n$ )
    for  $i = 2, 4, 6, \dots, n - 2$  do
        COMPARE-EXCHANGE( $a_i, a_{i+1}$ )
    end for
else
    COMPARE-EXCHANGE( $a_1, a_2$ )
end if

```

Algorithm 2 ODD-EVEN MERGE SORT

Input: Sequence $\mathbf{a}$ of length n (a power of 2).

Output: Sequence $\mathbf{a}$ is modified in-place to be sorted.

```

if  $n > 1$  then
    ODD-EVENMERGESORT( $a_1, a_2, \dots, a_{n/2}$ )
    ODD-EVENMERGESORT( $a_{n/2+1}, a_{n/2+2}, \dots, a_n$ )
    ODD-EVEN MERGE( $\mathbf{a}$ )
end if

```

The authors utilize **Batcher's Odd-Even Merge Sort**, a data-oblivious sorting network suitable for MPC environments.

- **Secure Comparison:** An optimized secure comparison protocol has been followed as described in

another research paper. Here, it is assumed to be a blackbox in which shares of two values are inserted as input and random shares of the minimum and maximum elements are given as output.

- **Data-Oblivious:** The sequence of comparisons is independent of input values, preventing side-channel leakage. Here, the position of the elements is being used rather than the values of the elements themselves.
- **Structure:** It is a sorting network built from fixed **Compare-Exchange** gates. For secret shares $[x]$ and $[y]$, the gate computes $[min]$ and $[max]$ using arithmetic operations on a securely computed comparison bit $[c]$ indicating whether $x < y$.
- **Complexity:** $\mathcal{O}(n \log^2 n)$ comparisons in $\mathcal{O}(\log^2 n)$ communication rounds.

2.3 Protocol & Scheme

- **Framework:** Implementation uses **Sharemind**, a general-purpose MPC platform.
- **Scheme:** **Additive Secret Sharing** over the ring $\mathbb{Z}_{2^{32}}$.
- **Security Model:** Information-theoretic security against a **passive (honest-but-curious) adversary** corrupting at most one of the three computing parties.

2.4 Results

The protocol was tested on a cluster of three commodity servers (Gigabit Ethernet).

- **Performance:** Successfully sorted 2^{14} **elements** in a little more than **3.5 minutes**. The implementation is moderately optimized, implying that this performance has scope for improvement.
- **Conclusion:** While slower than plaintext sorting, the method is feasible for high-latency batch processing tasks like inter-organizational log analysis.

3 Research Summary: Secure Sorting and Selection via Function Secret Sharing (FSS)

Amit Agarwal, Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Mahimna Kelkar, Yiping Ma (2024)

3.1 Premise

The paper addresses efficiency bottlenecks in secure multi-party sorting. While traditional Batcher’s networks require $\mathcal{O}(\log^2 n)$ rounds, this work utilizes **Function Secret Sharing (FSS)** to collapse operations into constant rounds. The core logic shifts

from a sequential network to a “Ranking and Routing” paradigm using succinct FSS keys.

- **Framework:** $(2 + 1)$ -party or 3-party honest-majority settings.

3.2 Core Technical Building Block: FSS

Function Secret Sharing (FSS) allows a function f from a known family to be shared into keys (k_0, k_1) . When a party P_i evaluates their key on an input x , they obtain a share y_i such that $y_0 + y_1 = f(x)$.

- **Distributed Comparison Function (DCF):** A specialized FSS where $f_{\alpha, \beta}(x)$ outputs β if $x < \alpha$, and 0 otherwise i.e. Let $f_{\alpha, \beta}^<(x) = \beta$ if $x < \alpha$ and 0 otherwise. This is the “engine” for secure comparisons.
- **Distributed Point Function (DPF):** A function that outputs non-zero only at a single point α . Let $f_{\alpha, \beta}^=(x) = \beta$ if $x = \alpha$ and 0 otherwise. This is used for oblivious indexing and routing.

3.3 Detailed Protocol Explanation

3.3.1 The CmpAgg (Compare-Aggregate) Protocol

Algorithm 3 FSS-based Ranking (CmpAgg)

Input: Secret shares $[x_1], \dots, [x_n]$
Setup: Dealer provides DCF keys $k_{i,j}$ for comparisons $x_i < x_j$

for each party $P_\sigma \in \{P_0, P_1\}$ **do**
 for $i, j \in \{1 \dots n\}$ **do**
 $[b_{i,j}]_\sigma \leftarrow \text{FSS.Eval}(k_{i,j}, [x_i]_\sigma, [x_j]_\sigma)$
 end for
 $[r_i]_\sigma \leftarrow \sum_{j=1}^n [b_{i,j}]_\sigma$
end for
Output: Secret shares of ranks $[r_1], \dots, [r_n]$

This is the primary protocol for ranking elements in a large domain. Instead of a sequential network of gates, it uses a parallel “all-to-all” comparison strategy that is made efficient through FSS.

- **Mechanism:** For a set of n secret-shared inputs $[x_1], \dots, [x_n]$, the parties use FSS to generate $\binom{n}{2}$ DCF keys. For every pair (x_i, x_j) , the DCF evaluates the comparison $x_i < x_j$.
- **Local Aggregation:** Since the outputs of the DCFs are additive secret shares, each party can locally sum the results. To find the rank of x_i , a party sums all shares resulting from the comparisons of x_i against every other x_j . Mathematically, The rank r_i for element x_i is defined as:

$$r_i = \sum_{j=1, j \neq i}^n \mathbf{1}_{x_j < x_i} + \sum_{j < i} \mathbf{1}_{x_j = x_i}$$

Where $\mathbf{1}$ is the indicator function computed via DCF.

- **Analysis:** This protocol requires only 1 online round to exchange the comparison results. Even though there are $O(n^2)$ logical comparisons, the communication scales only with the number of inputs n and the succinct FSS key size, rather than the total number of comparisons.

3.3.2 The mult-DPF Protocol (Small Domains)

When input domain is limited (e.g., 10-bit or 16-bit integers), the authors provide an even more efficient approach using DPFs that avoids the $O(n^2)$ overhead entirely. Typically, for domain size $M = 2^L$, we use the linearity of DPFs to build a shared histogram.

- **Mechanism:** Each party uses their shares of the inputs to evaluate a DPF. This creates a "shared histogram" of the entire dataset across the small domain. For each input x_i , parties evaluate a DPF $f_{x_i,1}^-(x)$. The collective histogram H is:

$$[H(v)] = \sum_{i=1}^n \text{DPF.Eval}(k_i, v) \quad \forall v \in \{0, \dots, M-1\}$$

where k_i is the succinct key of DPF $f_{x_i,1}^-(x)$.

- **Prefix Summation:** By performing a local prefix-sum on this shared histogram, the parties obtain the secret-shared rank for every possible value in the domain. The rank $[r_i]$ is then the prefix sum: $[r_i] = \sum_{v=0}^{x_i-1} [H(v)]$.
- **Result:** This protocol is exceptionally fast for "narrow" data (like ages, ratings, or small categories) because the communication depends only on the domain size and n , making it practically linear in many settings.

3.3.3 Shuffle Based Sorting and Selection

The original elements are secretly shuffled using a random permutation π as a part of pre-processing and then based on the shuffled list, rank vector $[v]$ is calculated using the ranking protocol described above. This rank vector $[v]$ is then made public. Trivial routing is done by each party using this rank vector $[v]$.

3.3.4 Three-Round Routing Protocol

Once the secret-shared ranks are computed, the protocol must move the payloads to their sorted positions without revealing the ranks to any party. The following 3-phase process utilizes the properties of Distributed Point Functions (DPF) to perform this oblivious permutation.

Algorithm 4 DPF-based Oblivious Routing

Inputs: Shared payloads $[z_i]$, Shared ranks $[y_i]$, Shared DPF keys $f_i(r_i) = 1$.

Round 2 (Public): Reconstruct $\hat{y}_i \leftarrow [y_i] + [r_i]$ for all i .

for each rank $k \in [n]$ **do**

for each element $i \in [n]$ **do**

$[e_{k,i}] \leftarrow \text{DPF.Eval}(f_i, \hat{y}_i - k)$

end for

$[\text{Output}_k] \leftarrow \sum_{i=1}^n [z_i] \cdot [e_{k,i}]$

end for

Output: Sorted shared sequence $[\text{Output}_1, \dots, \text{Output}_n]$.

• Phase 1: Pre-Processing

Let the shuffled list consist of elements where the i -th element has a secret-shared payload $[z_i]$ and a secret-shared rank $[y_i]$.

- **Random Masks:** For each index i , a random mask r_i is assigned and secret-shared among the parties.
- **DPF key:** For each index i , a DPF key is generated for the function $f_{r_i,1}^-(x)$. The keys for these n DPFs are distributed to the parties via FSS.

• Phase 2: Online Evaluation (Round 2)

Parties interact to compute the public shifted ranks, which allows for oblivious indexing.

1. **Masked Rank Reconstruction:** Each party computes their share $[y_i] + [r_i] = [\hat{y}_i]$. These shares are broadcasted and reconstructed so that $\hat{y}_i = y_i + r_i$ becomes public for all $i \in [n]$.
2. **Pointer Vector Construction:** For each destination index $k \in [n]$, parties evaluate their DPF shares at the shifted values. The share of the i -th component of the pointer vector for the k -th ranked element, denoted $[e_{k,i}]$, is computed as:

$$[e_{k,i}] = f_i(\hat{y}_i - k) \quad (1)$$

Summing these across parties results in a "one-hot" pointer vector e_k where the entry is 1 only if the i -th element's rank y_i is equal to k (i.e., when $\hat{y}_i - k = r_i$).

• Phase 3: Payload Selection (Round 3)

The final step maps the payloads to the sorted output using the pointer vectors.

- **Selection (Top- k):** To retrieve the payload of the element with rank k , parties compute the secret-shared dot product of the payload vector $\vec{z}$ and the pointer vector $\vec{e}_k$:

$$[\text{Sorted}_k] = \sum_{i=1}^n [z_i] \cdot [e_{k,i}] \quad (2)$$

- **Sorting:** For a full sort, the selection process is performed in parallel for every $k \in \{1, \dots, n\}$.

3.3.5 Two-Round Routing (DPF-based)

This protocol is just an improvement over the three-round routing protocol, which removes the necessity of calculating the ranks beforehand for DPF key calculations. The 2nd round maps elements to their sorted positions using a "blind" write mechanism.

- **Round 1:** The parties generate a random mask s_i and mask the elements with s_i .

$$x'_i = s_i \cdot x_i$$

Now the parties use a modified version of the *DPF.Eval* function wherein the function, now instead of producing 1 as the output, it produces s_i^{-1} . Which is later used in the "blind" write mechanism.

- **Round 2:** The authors use a DPF where the "point" α is the secret-shared rank. To place x_i at its rank r_i , we evaluate a DPF keyed at the secret value r_i to "pick" the correct element for each sorted position without ever learning what the ranks actually were. The j -th element of the sorted array Y is:

$$[y_j] = \sum_{i=1}^n \text{DPF.Eval}(k_{r_i}, j) \cdot [x'_i] \quad (3)$$

Since *DPF.Eval* results in shares of 1 if $r_i = j$ and 0 otherwise, the summation correctly selects the appropriate x_i . This "blind" routing allows the entire sort to complete in just 2 rounds (one for ranking, one for routing).

3.4 Implementation & Results

- **Implementation Settings:** The authors evaluated their protocols using a C++ implementation built on the *Emp-toolkit* and *Porthos* frameworks.
- **Performance:** Reaches the theoretical lower bound of **2 rounds** for sorting.
- **Key Result:** Communication costs are $O(n \cdot \lambda)$ where λ is the security parameter, providing up to 14× speedup over previous circuit-based approaches.

For sorting $n = 2^{10} = 1024$ secret shared 32-bit integers in a $2 + 1$ party semi-honest setting, this work shows the following results:

- **Rounds:** 2 – 3
- **Communication bandwidth:** 1.8 MB
- **Time taken in LAN setting:** 0.04s
- **Time taken in WAN setting:** 0.31s

4 Research Summary: Efficient Secure 3-Party Sorting with Applications

Gilad Asharov, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Ariel Nof, Benny Pinkas, Katsumi Takahashi, Junichi Tomida (2022)

4.1 Premise

The paper presents an efficient three-party sorting protocol secure against passive and active adversaries in the honest majority setting. The protocol can be combined with other secure protocols to enable various data analysis tasks.

- **MP Sorting:** The protocol securely sorts m shared items among three servers in the presence of at most one corrupted party without revealing the underlying values.
- **Private Set Intersection:** This protocol can be applied to *Private Set Intersection* problem and its modifications like *Threshold multi-party PSI* and *Data deduplication*.
- **Heavy Hitters & Telemetry:** Applied to privacy-preserving telemetry, clients share data, and servers identify the most frequent strings (e.g., heavy hitters) without leaking individual data.

4.2 Ideal Functionalities & Sub-protocols Used

4.2.1 $\mathcal{F}_{rand}$ - Generate shares of a random value

- **Functionality:** $\mathcal{F}_{rand}$ receives corrupt party P_i 's shares (α_i, α_{i+1}) . It generates a random $\alpha \in \mathcal{R}$ and sets $\alpha_{i-1} = \alpha - \alpha_i - \alpha_{i+1}$. It sends the respective shares to the honest parties.
- **Implementation:** It uses *Pseudorandom Secret Sharing (PRSS)*, where each pair of parties holds a shared PRF key. Thus, each P_i computes $\alpha_i := \text{PRF}_{key_i}(ctr)$ and $\alpha_{i+1} := \text{PRF}_{key_{i+1}}(ctr)$ for some online counter ctr .

4.2.2 $\mathcal{F}_{mult}$ - Multiplication

- **Functionality:** $\mathcal{F}_{mult}$ receives corrupt party P_i 's shares (c_i, c_{i+1}) as well each party's shares of a and b . It reconstructs a and b , computes $c = ab$ as well as $c_{i-1} = c - c_i - c_{i+1}$. It sends the respective shares to the honest parties.
- **Implementation:** It can be realized with passive security via the multiplication protocol of *High-Throughput Semi-Honest Secure Three-Party Computation with an Honest Majority* by Toshinori Araki, Jun Furukawa, Yehuda Lindell, Ariel Nof, Kazuma Ohara (2016)

4.2.3 Secure Shuffling Protocol

Algorithm 5 RESHARE

Input: A secret-shared value $\llbracket x \rrbracket$ with shares x_1, x_2, x_3 , a target party index i .

Output: Party P_i receives new randomized shares of x .

Parties P_{i-1} & P_{i+1} call $\mathcal{F}_{rand}$ to choose random values r_1, r_2 , and r_3 such that $r_1 + r_2 + r_3 = 0$.
 P_{i-1} & P_{i+1} locally compute $x'_1 := x_1 + r_1$, $x'_2 := x_2 + r_2$, and $x'_3 := x_3 + r_3$.
 P_{i-1} sends x'_i to P_i .
 P_{i+1} sends x'_{i+1} to P_i .

- In the REVEAL protocol, all parties collude together and share their secret shares of a value $\llbracket x \rrbracket$ with one another. Hence, all parties can compute the complete value x . In other words, x is publicly revealed or announced.

Algorithm 6 SHUFFLE

Input: A secret-shared permutation $\langle\langle \pi \rangle\rangle$ such that $\pi = \pi_3 \circ \pi_2 \circ \pi_1$, a shared vector $\llbracket \vec{a} \rrbracket$ such that $\vec{a} = (a_1, \dots, a_m)$.

Output: The secret-shared shuffled vector $\llbracket \vec{b} \rrbracket = \llbracket \pi(\vec{a}) \rrbracket$

Let $\vec{b}_0 = \vec{a}$.
for $i = 1$ **to** 3 **do**
 P_{i-1} and P_i define $\llbracket \vec{b}_i \rrbracket = \llbracket \pi_i(\vec{b}_{i-1}) \rrbracket$.
 The parties run RESHARE($\llbracket \vec{b}_i \rrbracket, i + 1$).
end for
The parties set $\vec{b} := \vec{b}_3$.
return $\llbracket \vec{b} \rrbracket$.

- **Communication Complexity:** Overall communication per party = $2m$ ring elements.
- The UNSHUFFLE protocol, as the name suggests, un-shuffles the shuffled vector $\llbracket \vec{b} \rrbracket = \llbracket \pi(\vec{a}) \rrbracket$. It's pseudocode is analogous to the SHUFFLE protocol.

4.3 Sorting Algorithm

The authors utilize a stable sort based on the **radix sort**, which is asymptotically better than previous secure sorting protocols. Instead of shuffling the entire length of the items after each comparison step, it computes a composition of permutations.

4.3.1 Characteristics & Primitive

- **Data-Oblivious:** The sequence of comparisons is independent of input values, preventing side-channel leakage. Instead of using fixed routing paths, obliviousness is achieved by securely computing secret-shared permutations bit-by-bit.
- **Stable Sorting:** Unlike other sorting algorithms used in secure MP sorting protocols, radix sort provides a stable sorting of elements.

Algorithm 7 GENBITPERM - Generating permutation of stable sort for a single bit key

Input: Secret-shared bit-wise keys $\llbracket \vec{k} \rrbracket$, where $k_i \in \{0, 1\} \forall i \in \{1, \dots, m\}$.

Output: The secret-shared permutation $\llbracket \vec{\rho} \rrbracket$ which is a stable sorting of $\vec{k}$

Complexity per party: $m||\mathcal{R}||$ bits

for $1 \leq i \leq m$ **do**
 $\llbracket f_i^{(0)} \rrbracket := 1 - \llbracket k_i \rrbracket$.
 $\llbracket f_i^{(1)} \rrbracket := \llbracket k_i \rrbracket$.
end for
 $\llbracket s \rrbracket := \llbracket 0 \rrbracket$.
for $j = 0$ **to** 1 **do**
 for $i = 1$ **to** m **do**
 $\llbracket s \rrbracket := \llbracket s \rrbracket + \llbracket f_i^{(j)} \rrbracket$.
 $\llbracket s_i^{(j)} \rrbracket := \llbracket s \rrbracket$
 end for
end for
for $1 \leq i \leq m$ **do**
 Send $(\llbracket k_i \rrbracket, \llbracket s_i^{(1)} \rrbracket - \llbracket s_i^{(0)} \rrbracket)$ to $\mathcal{F}_{mult}$ and receive $\llbracket t_i \rrbracket$.
 $\llbracket \rho(i) \rrbracket := \llbracket s_i^{(0)} \rrbracket + \llbracket t_i \rrbracket$
end for
return $\llbracket \vec{\rho} \rrbracket = (\llbracket \rho(1) \rrbracket, \dots, \llbracket \rho(m) \rrbracket)$

Algorithm 8 COMPOSE - Composition of two share-vector permutations

Input: Secret-shared permutations $\llbracket \vec{\sigma} \rrbracket, \llbracket \vec{\rho} \rrbracket$.

Output: Secret-shared permutation $\llbracket \vec{\tau} \rrbracket$, where $\tau = \rho \circ \sigma$.

Complexity per party: $5m||\mathcal{R}||$ bits

Call $\mathcal{F}_{rand}$ and obtain $\langle\langle \pi \rangle\rangle$.
 $\llbracket \pi(\vec{\sigma}) \rrbracket \leftarrow \text{SHUFFLE}(\langle\langle \pi \rangle\rangle, \llbracket \vec{\sigma} \rrbracket)$
Call REVEAL($\llbracket \pi(\vec{\sigma}) \rrbracket$) to obtain $\pi(\vec{\sigma}) = \sigma \circ \pi^{-1}$.
Locally apply $(\sigma \circ \pi^{-1})^{-1} = \pi \circ \sigma^{-1}$ to $\llbracket \vec{\rho} \rrbracket$ to obtain $\llbracket \vec{\gamma} \rrbracket = \llbracket \pi \circ \sigma^{-1}(\vec{\rho}) \rrbracket$.
 $\llbracket \vec{\tau} \rrbracket \leftarrow \text{UNSHUFFLE}(\langle\langle \pi \rangle\rangle, \llbracket \vec{\gamma} \rrbracket)$
return $\llbracket \vec{\tau} \rrbracket$

Algorithm 9 APPLYPERM - Applying a shared-vector permutation

Input: A secret-shared permutation $\llbracket \vec{\rho} \rrbracket$, a secret-shared vector $\llbracket \vec{k} \rrbracket$.

Output: A secret-shared vector $\llbracket \vec{k}' \rrbracket$ such that $\vec{k}' = \rho(\vec{k})$.

Complexity per party: $5m||\mathcal{R}||$ bits

Call $\mathcal{F}_{rand}$ and receive $\langle\langle \pi \rangle\rangle$.
 $\llbracket \pi(\vec{\rho}) \rrbracket \leftarrow \text{SHUFFLE}(\langle\langle \pi \rangle\rangle, \llbracket \vec{\rho} \rrbracket)$
 $\llbracket \pi(\vec{k}) \rrbracket \leftarrow \text{SHUFFLE}(\langle\langle \pi \rangle\rangle, \llbracket \vec{k} \rrbracket)$
Call REVEAL($\llbracket \pi(\vec{\rho}) \rrbracket$) to obtain $\pi(\vec{\rho}) = \rho \circ \pi^{-1}$.
Locally apply $\rho \circ \pi^{-1}$ on $\llbracket \pi(\vec{k}) \rrbracket$ to obtain $\llbracket \vec{k}' \rrbracket = \llbracket \rho(\vec{k}) \rrbracket$.
return $\llbracket \vec{k}' \rrbracket$

- **Structure:** The algorithm is built on bitwise radix sort processing starting from the rightmost LSB. It uses a secure COMPOSE protocol to mathematically merge the routing instructions of every single bit into one single *master permutation*.

Algorithm 10 GENPERM - Securely Generating a Stable Sorting Permutation

Input: Secret-shared keys $\llbracket \vec{k} \rrbracket = (\llbracket k_1 \rrbracket, \dots, \llbracket k_m \rrbracket)$ where $\llbracket k_i \rrbracket$ has bits $(\llbracket k_i^{(1)} \rrbracket, \dots, \llbracket k_i^{(l_k)} \rrbracket)$.

Output: Permutation $\llbracket \vec{\sigma} \rrbracket$ for stable sorting $\vec{k}$

Complexity per party: $11m \cdot \|\mathcal{R}\| \cdot l_k$ bits

For each $j \in [l_k]$, let $\llbracket \vec{k}^{(j)} \rrbracket = (\llbracket k_1^{(j)} \rrbracket, \dots, \llbracket k_m^{(j)} \rrbracket)$.

$\llbracket \vec{\rho}_1 \rrbracket \leftarrow \text{GENBITPERM}(\llbracket \vec{k}^{(1)} \rrbracket)$

$\llbracket \vec{\sigma}_1 \rrbracket := \llbracket \vec{\rho}_1 \rrbracket$

for $j = 2$ **to** l_k **do**

$\llbracket \vec{k}'^{(j)} \rrbracket \leftarrow \text{APPLYPERM}(\llbracket \vec{\sigma}_{j-1} \rrbracket, \llbracket \vec{k}^{(j)} \rrbracket)$

$\llbracket \vec{\rho}_j \rrbracket \leftarrow \text{GENBITPERM}(\llbracket \vec{k}'^{(j)} \rrbracket)$

$\llbracket \vec{\sigma}_j \rrbracket \leftarrow \text{COMPOSE}(\llbracket \vec{\sigma}_{j-1} \rrbracket, \llbracket \vec{\rho}_j \rrbracket)$

end for

return $\llbracket \vec{\sigma}_{l_k} \rrbracket$

- **Overall communication complexity per party:** $11m \cdot \|\mathcal{R}\| \cdot l_k + 3m \cdot \|\mathcal{R}\| + 2m \cdot \|\mathcal{R}'\|$ bits
(Upper bound = $14l_k \cdot m \log m + 2l_p m$ bits)

4.3.2 $\mathcal{F}_{\text{sort}}$ - Stable sorting functionality

- **GenPerm Functionality:** $\mathcal{F}_{\text{sort}}$ receives all shares of $\llbracket \vec{k} \rrbracket$ from the honest parties. It reconstructs $\vec{k}$ and computes the permutation σ which corresponds to the stable sorting of $\vec{k}$. It then secret shares $\llbracket \sigma \rrbracket$ amongst the parties.
- **ApplyPerm Functionality:** $\mathcal{F}_{\text{sort}}$ receives all shares of $\llbracket \sigma \rrbracket$ and $\llbracket \vec{v} \rrbracket$ from the honest parties. It reconstructs σ and $\vec{v}$ and computes $\vec{v}' = \sigma(\vec{v})$. It then secret shares $\llbracket \vec{v}' \rrbracket$ amongst the parties.

4.4 Modifications for Malicious Security

- **Authenticated secret sharing:** The protocol is modified by adding a MAC to each secret, meaning parties hold $(\llbracket x \rrbracket, \llbracket r \cdot x \rrbracket)$ for a random, unknown r . All operations done on $\llbracket x \rrbracket$ will also be done on $\llbracket r \cdot x \rrbracket$. The MAC will be used for verification to detect cheating at the end of any operation.
- **Security upto additive attacks:** Semi-honest protocols are modified to be secure up to additive attacks, where a malicious adversary can only introduce an independent offset error to the output.
- $\mathcal{F}_{l\text{-mult}}^{\text{add}}$ **functionality:** An extended inner-product functionality that incorporates an additive error Δ chosen by the adversary.
- $\mathcal{F}_{\text{reshare-shuffle}}^{\text{add}}$ **functionality:** Models resharing after a local shuffle, allowing a corrupted party to inject an additive error into their reshared output. Even though it cannot guarantee the correctness of output in presence of malicious adversary, the verification of MAC tags at

the end of the protocol will ensure that cheating is detected.

• Modifications to the semi-honest protocol:

1. *Input randomization:* The original input $\llbracket \vec{k} \rrbracket$ is pre-processed to attach a MAC with every input element. Thus, the new input becomes $(\llbracket \vec{k} \rrbracket, \llbracket r \cdot \vec{k} \rrbracket)$.
2. *Modified GENPERM:* The parties run GENPERM on both $\llbracket \vec{k} \rrbracket$ and $\llbracket r \cdot \vec{k} \rrbracket$. Each $\mathcal{F}_{\text{mult}}$ call is replaced with $\mathcal{F}_{l\text{-mult}}^{\text{add}}$ with $l = 1$. Every SHUFFLE call is accompanied by an $\mathcal{F}_{\text{reshare-shuffle}}^{\text{add}}$ call. Whenever there is a REVEAL call, the parties first run the verification protocol.

4.5 Implementation & Results

- **Implementation Settings:** The protocols were implemented in **C++11**, taking advantage of multi-threading and AES-NI on **AWS c5.18xlarge** servers.
- **Scheme: Linear (additive) secret sharing** over the ring $\mathbb{Z}_p$ ($p = 2^{61} - 1$).
- **Results:** The implementation proves concretely efficient, sorting one million 32-bit records in 2.26 seconds with semi-honest security and 5.89 with malicious security ($\lambda = 32$).

4.6 Improving Concrete Efficiency

- **Batch processing:** Concrete efficiency is improved by processing multiple bits at a time (e.g., $L = 3$) in GENMULTIBITSORT, reducing round complexity.
- **Reusing permutations:** The same random permutations can be reused across APPLYPERM and COMPOSE steps within a single iteration to minimize communication overhead.
- **Permuting small shares:** Communication is reduced by operating on small shares in $\mathbb{Z}_2$ rather than a large ring during APPLYPERM, utilizing modulus conversion before proceeding.
- **Optimized shuffling:** P_1 knows π_2 and π_1 that used for the first and second shuffling steps. The randomness used in resharing also can be obtained via $\mathcal{F}_{\text{rand}}$ beforehand. Therefore, P_1 can permute $\vec{a}$ using π_2 and π_1 at once.

5 Research Summary: PRAC: Round-Efficient 3-Party MPC for Dynamic Data Structures

Sajin Sasy, Adithya Vadapalli, Ian Goldberg (2023)

5.1 Premise

The paper presents **Private Random Access Computations (PRAC)**, a 3-party Secure Multi-Party Computation (MPC) framework designed to support random-access data structure algorithms with efficient communication costs in terms of both rounds and bandwidth. PRAC achieves algorithmic improvements by exploiting the observation that *a secure protocol can gain efficiency if it explicitly reveals information inherently leaked by the algorithm.*

- **Oblivious Binary Search:** An optimized protocol for static data that utilizes *Incremental Distributed Point Functions (IDPFs)* to explicitly leak access patterns. This reduces the bandwidth cost from $\mathcal{O}(\log^2 n)$ to $\mathcal{O}(\log n)$ for obliviously searching over n items.
- **Oblivious Heaps:** A restrictively dynamic data structure supporting arbitrary insertions and minimum element deletions. Its implementation leverages optimized binary search to reduce insertion communication rounds from $\mathcal{O}(\log n)$ to $\mathcal{O}(\log \log n)$. Additionally, EXTRACT-MIN’s bandwidth is reduced to $\mathcal{O}(\log n)$ using *Wide DPFs (WDPFs)*.
- **Oblivious AVL Trees:** The first oblivious AVL tree protocol for MPC in a distributed trust setting, enabling a fully dynamic data structure. It supports arbitrary insertions and deletions with $\mathcal{O}(\log n)$ rounds and bandwidth, ensuring no party learns the underlying data or the tree’s structure.

5.2 Distributed Point Functions (DPFs) and Variations

5.2.1 Distributed Point Functions (DPFs)

- **Description:** A DPF is represented as two complete binary trees of height $h = \log n$, where each computing party learns exactly one of the trees.
- **Construction:** Each node begins within the tree contains a λ -bit label and a single advice/flag bit. The construction begins with the roots of the two trees having random λ -bit labels. An invariant, where corresponding nodes in the two trees are identical except for one on each level, is maintained through the use of h correction words, one for each non-leaf level.

- **Evaluation:** Evaluating the DPF involves traversing the path from the root to a specific leaf i , applying the necessary correction words along the way and outputting the final label on the leaf.

5.2.2 Incremental DPFs (IDPFs)

- **Description:** IDPFs leverage the tree-based construction to allow parties to evaluate their DPFs at bitstrings of any length $1 \leq j \leq \log n$.
- When evaluated, the parties will produce different outputs exactly when the input string is a prefix of the target index i^* , and will output the same value otherwise.
- Although constructing an IDPF costs roughly 50% more than a regular DPF, it is highly efficient because it effectively yields $\log n$ DPFs whose target indices are the prefixes of a single target index.

5.2.3 Wide DPFs (WDPFs)

- **Description:** WDPFs are a variation where the leaf nodes are significantly wider than the internal nodes of the tree.
- For a WDPF of width w , the leaf nodes are exactly w times longer than the internal nodes.
- This is achieved by applying a “length- w -stretching” Pseudorandom Generator (PRG) right before applying the final correction word, which itself is w times longer.
- While highly useful for bulk related-index updates, creating a WDPF of width w requires $n \cdot (w - 1)$ more AES evaluations than a standard DPF.

5.3 Distributed Oblivious RAMs (DORAMs)

- **Core Concept:** DORAMs allow multiple computing servers to store and process data collectively without any single server ever knowing the data contents or access patterns.
- Two main categories - *communication-efficient* and *computation-efficient*.

5.3.1 PRAC’s Approach (DUORAM)

- PRAC uses DUORAM (communication-efficient) because it has near-zero initialization costs and stores memory efficiently as secret shares.
- It shifts the heavy lifting like generating DPFs to a preprocessing phase, making live online operations extremely fast (constant rounds/bandwidth).

- **PRAC** extends the state-of-the-art DUORAM, providing more flexibility in how memory is shared (supporting XOR- or additive-shared indices and memory in any combination).

5.4 Oblivious Binary Search

- **Core Premise:** The protocol searches a sorted, additive-shared Distributed ORAM (DORAM) for a specific item. It returns the secret-shared index of the smallest memory location containing a value at least as large as the target item.
- **Basic Approach:** It executes a standard binary search algorithm using general-purpose DORAM reads at each step. It requires $\log n$ separate DORAM read operations, which consequently requires generating $\log n$ independent Distributed Point Functions (DPFs).
- **Inefficiency:** This basic method over-hides information. It completely conceals the accessed indices, even though the structural path of a binary search (jumping left or right by predictable powers of 2) is already public knowledge.
- **Optimized Binary Search (PRAC's Approach):** PRAC explicitly leaking the inherently known structural access pattern of the search. It replaces the $\log n$ standard DPFs with exactly one Incremental DPF (IDPF). The optimization works because the sequence of accessed indices forms an incrementally growing prefix (appending a bit at each depth based on the comparison result), which perfectly matches the structure of an IDPF. Unlike the basic version, the optimized protocol completely eliminates the need for expensive Flag-Word MPC multiplications.
- **Performance Gains:** This algorithmic improvement reduces the communication bandwidth overhead from $\mathcal{O}(\log^2 n)$ to $\mathcal{O}(\log n)$ and lowers computation from $\mathcal{O}(n \log n)$ to $\mathcal{O}(n)$.

5.5 Oblivious Heaps

- **Core Concept:** PRAC implements a restrictively dynamic heap data structure. It supports arbitrary insertions (HEAPINSERT) and the deletion of the minimum element at the root (EXTRACTMIN). The heap utilizes an implicit structure where the root is at index 1, the left child is at $2i$, and the right child is at $2i + 1$.

5.5.1 Basic Oblivious Protocols

- **Basic HEAPINSERT:** Adds the new item to the end of the heap array and obviously compares and swaps it with its parent all the way up to the root to restore the heap property. This requires $\mathcal{O}(\log n)$ sequential oblivious comparisons and swaps, but notably requires zero DORAM

operations.

- **Basic EXTRACTMIN:** Saves the root, replaces it with the last leaf node, and trickles it down using a HEAPIFY algorithm. At each level, it obviously compares the node with its children and swaps it with the smaller one.
- **The Inefficiency:** The basic EXTRACTMIN requires three independent oblivious reads and three updates at each level using a general-purpose DORAM. This is inefficient because it hides the inherently public relationship between the accessed indices (parent, left child, right child).

5.5.2 Optimized Protocols

- **Optimized HEAPINSERT:** The specific path from the root down to the newly added empty node is public knowledge, and this path is already sorted. PRAC uses the optimized binary search protocol (which uses a single IDPF of height $\log \log n$) to quickly find the exact position on this path where the new item belongs. Once the position is found, PRAC uses parallel Flag-Word multiplications to obviously shift all larger elements down one spot and write the new item into the resulting hole, drastically reducing communication rounds.
- **Optimized EXTRACTMIN:** PRAC explicitly reveals that the algorithm is reading related indices (node and its two children) at each level. It uses a single Wide Incremental DPF to execute these reads and updates simultaneously. Because a WDPF's leaf stores three random values instead of one, it can update the parent, left child and right child in the accessible sets using the exact same DPF.

5.6 AVL Trees

- **Explicit Structure:** Unlike heaps and binary search arrays which use an implicit tree structure dictated by memory indices, AVL trees use an explicit structure. Nodes are stored arbitrarily in the DORAM, and the tree structure is maintained dynamically via explicit child pointers to avoid the prohibitive worst-case cost of oblivious reorganization during rebalancing.
- **Oblivious Insertion (Delayed Balancing):** Instead of performing oblivious rotations at all levels of the tree to hide the depth of an imbalance, the AVL insert utilizes a novel recursive insert protocol. This protocol returns an insertion state, st_I , that captures the exact imbalance point (if any) during the insertion. A subsequent FixImb protocol is then executed to perform necessary rotations obviously and update balance

bits, arriving at a balanced state.

- **Oblivious Deletion:** Deletions are processed by obliviously handling three cases: (i) if the node to be deleted has no children, it is deleted directly, (ii) if it has one child, the node is deleted and its parent is updated to point to the only child, and (iii) if it has both children, it is swapped with its successor first and then deleted. Unlike insertion, deleting a node can require imbalance-fixing rotations at each level on the deletion path, requiring FixImb to be performed at every level.

5.7 Implementation & Results

The PRAC framework was evaluated with a reference implementation in C++ using Boost Asio, simulated under realistic Internet conditions (30 ms latency, 100 Mbps throughput).

- **Binary Search Performance:** The optimized binary search provides $> 27\times$ improvements in wall-clock time and $> 3\times$ reduction in bandwidth over other approaches for a memory with 2^{26} items.
- **Heap Performance:** For the same 2^{26} item setting, the heap’s Extract-Min protocol achieves a $> 31\times$ speedup in wall-clock time and a $> 13\times$ reduction in bandwidth.
- **Conclusion:** By shifting to an explicit structure and intelligently revealing inherently leaked access patterns, PRAC dramatically improves the viability of complex, dynamic data structures like AVL trees and heaps in 3-party distributed-trust environments.

6 Research Summary: Graphiti: Secure Graph Computation

Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, Bhavish Raj Gopal (2024)

6.1 Premise

The paper presents Graphiti, a highly scalable framework for secure graph computation that improves upon prior state-of-the-art systems (like GraphSC). It allows mutually distrusting parties to operate on graphs without leaking sensitive topological or node-level data.

- **$\mathcal{O}(1)$ Round Complexity:** Unlike previous frameworks whose round complexity scales linearly with the graph size $N = |V| + |E|$, Graphiti’s message-passing round complexity is strictly constant and independent of N .
- **Decoupling Scatter:** The protocol fundamentally optimizes the message-passing phase by decoupling the traditional SCATTER primitive into two distinct operations: PROPAGATE (distribut-

ing data) and APPLYE (updating edges)

- **Linear Operations via Prefix Sums:** It completely removes the need for interactive oblivious selection (which requires heavy secure multiplications) by replacing them with non-interactive cumulative prefix sums over specifically ordered lists.
- **Applications:** Existing applications like *BFS*, *histograms* and *matrix factorisation*, *computing PageRank* and its variants, *graph convolutional network evaluation*, etc. can directly be improved by using Graphiti as the underlying secure graph computation framework.

6.2 Ideal Functionalities and Sub-Protocols Used

6.2.1 $\mathcal{F}_{Shuffle}$ - Secure Shuffling Functionality

- **Functionality:** Takes as input a secret-shared vector from the computing parties, applies a random secret permutation π , and returns the secret-shared shuffled vector.
- **Implementation:** Graphiti introduces a novel $(2 + 1)$ -Shuffle protocol tailored for the 2-party setting with a trusted helper (preprocessing), executing in exactly 1 online round.

6.2.2 $\mathcal{F}_{insec.Sort}$ - Insecure Sorting Functionality

- **Functionality:** Generates a public mapping (permutation) that transitions a randomly shuffled DAG-list into a specific sorted order. Because the input array is already randomly shuffled by $\mathcal{F}_{Shuffle}$, revealing this mapping leaks no topological information.

6.2.3 Secure Shuffling Protocol: $(2 + 1)$ -Shuffle (Online Phase)

To efficiently transition between the vertex, source and destination ordering, Graphiti requires a secure shuffle. The authors design a novel $\Pi_{(2+1)-Shuffle}$ protocol for two computing parties (P_0, P_1) and one trusted helper (P_2) that operates strictly in the offline preprocessing phase.

- **Input:** Additive shares $\langle T \rangle_0, \langle T \rangle_1$ of table T .
- **Preprocessing:** The helper party P_2 samples the composite permutations and generates random masking vectors (R_0, R_1, B_0, B_1) . These are distributed to P_0 and P_1 offline.
- **Output:** Shares of the shuffled table $\langle T_O \rangle_0, \langle T_O \rangle_1$ where $T_O = \pi(T)$.

• The Protocol:

1. P_1 computes and sends $A_0 = \pi_1(\langle T \rangle_1 + R_1)$ to P_0 .
2. P_0 computes and sends $A_1 = \pi'_0(\langle T \rangle_0 + R_0)$ to P_1 .
3. P_0 computes $\langle T_O \rangle'_0 = \pi_0(A_0)$ and sets $\langle T_O \rangle_0 = \langle T_O \rangle'_0 - B_0$.
4. P_1 computes $\langle T_O \rangle'_1 = \pi'_1(A_1)$ and sets $\langle T_O \rangle_1 = \langle T_O \rangle'_1 - B_1$.
5. P_0 and P_1 locally sample a shared random permutation π_2 and apply it to their respective shares to mask the final output from the helper.

- **Communication Complexity:** $2N\ell$ bits (exactly 1 message of size $N\ell$ per party in the online phase).

6.3 Graphiti Algorithm and Framework

6.3.1 DAG-List Representation

The graph $G(V, E, data)$ is represented as a list, called the DAG-list, which combines both vertices and edges into a single unified structure. This ensures the topology remains hidden during linear scans.

- Each entry is encoded as a tuple: **(src, dst, isV, data)**.
- A node u is encoded as $(u, u, 1, data)$.
- A directed edge $e(u, v)$ is encoded as $(u, v, 0, data)$.
- The data component is logically split during message passing into $data_s$ (data to send), $data_r$ (data received by an edge), and $data_g$ (data gathered by a node).

6.3.2 Core Message-Passing Primitives

- **Propagate:** A node propagates its state data to its outgoing edges. Through this operation, each directed edge $e(u, v)$ updates its data component as $e.data_r = e.data_r \parallel u.data_s$.
- **ApplyE (Apply-Edge):** All edges locally update their data by applying a user-defined function f_{AE} .
- **Gather:** A destination node aggregates the propagated data from all its incoming edges using a linear aggregation function $\oplus$. $v.data_g = v.data_g \parallel \bigoplus_{e(u,v)} e.data_r$.
- **ApplyV (Apply-Vertex):** All nodes locally update their state data by applying a user-defined function f_{AV} based on the aggregated results.

6.3.3 Characteristics

- **Data-Oblivious Prefix Sums:** Edge propagation and node aggregation are achieved using a cumulative sum. Because addition is a linear operation in secret sharing, this requires zero online interaction.
- **Structure:** Each message-passing iteration consists of:

PROPAGATE $\rightarrow$ APPLYE $\rightarrow$ GATHER $\rightarrow$ APPLYV

Algorithm 11 PROPAGATE

Input: DAG-list G in *vertex order*

Output: Secret-shared DAG-list where node data is propagated to outgoing edges

Complexity Per Party: $O(1)$ rounds

for $i = |V|$ **to** 2 **do**

$G[i].data_r \leftarrow G[i].data_s - G[i-1].data_s$

end for

Transition to the source order of the DAG-list via $\mathcal{F}_{Shuffle}$ and public mappings.

for $i = |N|$ **to** 1 **do**

$G[i].data_r \leftarrow \sum_{j=1}^i G[j].data_r - G[i].data_s$

end for

Algorithm 12 GATHER

Input: DAG-list G in *destination order*

Output: Secret-shared DAG-list where incoming edge data is aggregated into destination nodes

for $i = 1$ **to** N **do**

$G[i].data_g \leftarrow \sum_{j=1}^i G[j].data_r$

end for

Transition to the vertex order of the DAG-list via $\mathcal{F}_{Shuffle}$ and public mappings.

for $i = |N|$ **to** 2 **do**

$G[i].data_g \leftarrow G[i].data_g - G[i-1].data_g$

end for

6.4 Implementation & Results

- **Implementation Settings:** Developed in C++ leveraging the EMP-toolkit, executed over a simulated 1 Gbps LAN on AMD Ryzen Threadripper PRO servers.
- **Scheme:** 2-Party Computation (2PC) over rings with a Trusted Helper operating strictly in the offline preprocessing phase (secure against a semi-honest adversary).
- **Results:** The framework achieved massive scalability. Running a 10-hop Breadth-First Search (BFS) for contact tracing on a graph with 10^7 elements completed in under 2 minutes. This yielded up to a 1034x speedup in online runtime and a 106x reduction in communication costs compared to the GraphSC baseline.

6.5 Improving Concrete Efficiency

- **Optimized ApplyV:** Because Graphiti always returns the DAG-list to the vertex Order at the end of GATHER, the non-linear APPLYV step (e.g., threshold comparisons) only needs to be executed on the first $|V|$ elements, rather than the entire list of size N .
- **Amortized Shuffling:** The mappings between vertex, source and destination orders remain static across iterations. These transitions are pre-computed during an initialization phase, meaning multiple message-passing rounds simply reuse the same permutations.
- **Minimal Data Reordering:** Instead of reshuffling the entire DAG-list tuples (which include static topological metadata), Graphiti only reorders the specific data payloads during transitions, massively reducing communication bandwidth.

7 Extending PRAC: Oblivious Breadth-First Search (OBFS)

7.1 Introduction

Breadth-First Search (BFS) is a fundamental graph traversal algorithm. However, executing BFS in a Secure Multi-Party Computation (MPC) setting introduces severe privacy risks, primarily **topology leakage**. *Standard pointer-chasing* and *dynamic queue structures* inherently leak the degrees of individual vertices and the overall diameter of the graph through memory access patterns.

This report details a novel **Oblivious Breadth-First Search (OBFS) protocol** tailored for the **PRAC (Private Random Access Computations) 3-party framework**. By flattening the graph representation and replacing dynamic pointer-chasing with data-independent Oblivious Bitonic Sorts and linear scans, this protocol guarantees zero topology leakage while optimizing for network round efficiency.

7.2 Data Structure: The Graph Array

To prevent degree leakage without incurring the massive memory overhead of dummy-edge padding (where every node is padded to a maximum degree D_{max}), the graph is stored as a tightly packed, single-dimensional array.

The graph is represented as a secret-shared Distributed ORAM (DORAM) array G of size $N = V + E$. There is no physical separation between vertex records and edge records. Each element $G[i]$ is a

secret-shared struct containing the following fields, where the computing parties hold either additive shares (AS) or boolean shares (BS):

$$\bullet \text{ } isEdge^{BS} = \begin{cases} 1 & \text{if struct corresponds to edge} \\ 0 & \text{if struct corresponds to vertex} \end{cases}$$

$$\bullet v_1^{AS} = \begin{cases} i & \text{if } isEdge = 0 \text{ and vertex no.} = i \\ i & \text{if } isEdge = 1 \text{ and edge} = (i, j) \end{cases}$$

(Source vertex)

$$\bullet v_2^{AS} = \begin{cases} i & \text{if } isEdge = 0 \text{ and vertex no.} = i \\ j & \text{if } isEdge = 1 \text{ and edge} = (i, j) \end{cases}$$

(Destination vertex)

$$\bullet q^{BS} = \begin{cases} 1 & \text{if the vertex/edge is currently being} \\ & \text{explored or part of the queue} \\ 0 & \text{otherwise} \end{cases}$$

$$\bullet visited^{BS} = \begin{cases} 1 & \text{if the vertex/edge was placed} \\ & \text{in the queue in current or any} \\ & \text{previous step} \\ 0 & \text{otherwise} \end{cases}$$

- key^{AS} (Transient sorting key used strictly during the routing phases)

7.3 Cryptographic Primitives

The protocol leverages the following highly optimized PRAC primitives to bypass the need for heavy DUORAM index masking:

1. **MPC AND/OR:** Parties P_0 and P_1 hold boolean shares of $x \in 0,1$ and $y \in 0,1$, and wish to jointly compute boolean shares of the AND of x and y . We denote the operation as $z^{BS} \leftarrow x^{BS} \wedge y^{BS}$. (MPC OR can be realized using the MPC AND primitive, since $A \vee B = A + B - A \wedge B$)
2. **MPC Flag-Word (FW) Multiplication:** P_0 and P_1 hold additive shares of $y \in \mathbb{Z}_{2^r}$ and boolean shares of $f \in 0,1$. The two parties compute additive shares of $z \in \mathbb{Z}_{2^r}$, such that $z = f \cdot y$. We denote this as $z^{AS} \leftarrow f^{BS} \cdot y^{AS}$.
3. **Oblivious Select:** P_0 and P_1 hold additive shares of both x and y . Oblivious Select, denoted as OSELECT, is defined as $z^{AS} \leftarrow \text{OSELECT}(x^{AS}, y^{AS}, b^{BS})$. If $b = 1$, then $z = y$ and if $b = 0$, then $z = x$.
4. **Oblivious Bitonic Sort:** PRAC provides an implementation of bitonic sort, which will obliviously sort N items in $\mathcal{O}(\log^2 N)$ rounds.

7.4 The PRAC - Secure BFS Protocol

To completely hide the graph's depth, the outer BFS loop executes a strict worst-case bound of $V - 1$ iterations. Each iteration consists of two main phases that route the *queue status* through the graph without relying on pointers.

7.4.1 Initialization

The graph is loaded into G . For the starting (source) vertex S :

- $q^{BS} = 1$
- $visited^{BS} = 1$

For all other vertices and all edges, $q^{BS} = 0$, and $visited^{BS} = 0$.

7.4.2 Phase 1: Source-to-Edge Propagation

The goal is to pass the queue flag from vertices to their outgoing edges.

1. **Prepare Sort Key:** For every element $i \in [1, N]$:

$$G[i].key^{AS} \leftarrow G[i].v_1^{AS}$$

2. **Oblivious Sort:** The array G is sorted using oblivious bitonic sort provided by PRAC.

- *Primary Criterion:* key^{AS} (ascending)
- *Secondary Criterion:* $isEdge^{BS}$ (ascending)

Result: Every vertex struct is immediately followed by its outgoing edge structs.

3. **Forward Scan (Trickle-Down):** A linear scan propagates the queue flag using a secret-shared running register t_1^{BS} , initialized to 0. The intuition is that *if a vertex is currently in the queue, all its outgoing edges must also be inserted in the queue.*

7.4.3 Phase 2: Edge-to-Destination Resolution

The goal is to pass the queue status from the edges to their destination vertices to form the next iteration's queue, resolving write-collisions obliviously.

1. **Prepare Sort Key:**

For every element $i \in [1, N]$:

$$G[i].key^{AS} \leftarrow G[i].v_2^{AS}$$

2. **Oblivious Sort:** The array G is sorted again using oblivious bitonic sort provided by PRAC.

- *Primary Criterion:* key^{AS} (ascending)
- *Secondary Criterion:* $type^{BS}$ (descending)

Result: All incoming edge structs appear immediately before their corresponding destination vertex struct.

3. **Forward Scan (Accumulate and Apply):** A linear scan accumulates active edges and activates unvisited target vertices using another secret-shared running registers t_2^{BS} and t_3^{BS} , initialized to 0. The intuition is that *if any one of the incoming edges is currently in the queue, the destination vertex must be inserted in the queue.*

4. **Edge Cleanup:** Obviously clear all edge frontier flags for the next iteration.

Algorithm 13 Phase 1: Source-to-Edge Propagation (Trickle-Down)

Input: Graph array G of size $N (= V + E)$

for $i = 1$ **to** N **do**
 $G[i].key^{AS} \leftarrow G[i].v_1^{AS}$
end for

$OSORT(G, key, isEdge)$

$t_1^{BS} \leftarrow 0$
for $i = 1$ **to** N **do**
 $t_1 \leftarrow OSELECT(G[i].q, t_1, G[i].isEdge)$
 $G[i].q \leftarrow OSELECT(G[i].q, t_1, G[i].isEdge)$
end for

Algorithm 14 Phase 2: Edge-to-Destination Resolution

Input: Graph array G of size $N (= V + E)$

for $i = 1$ **to** N **do**
 $G[i].key^{AS} \leftarrow G[i].v_2^{AS}$
end for

$OSORT(G, key, isEdge)$

$t_2^{BS} \leftarrow 0, t_3^{BS} \leftarrow 0$
for $i = 1$ **to** N **do**
 $t_2^{BS} \leftarrow OSELECT(t_2^{BS}, t_2^{BS} \vee G[i].q^{BS}, G[i].isEdge^{BS})$
 $t_3^{BS} \leftarrow OSELECT(t_2^{BS} \wedge \neg G[i].visited^{BS}, t_3^{BS}, G[i].isEdge^{BS})$
 $G[i].q^{BS} \leftarrow OSELECT(t_3^{BS}, G[i].q^{BS}, G[i].isEdge^{BS})$
 $G[i].visited^{BS} \leftarrow OSELECT(G[i].visited^{BS} \vee t_3^{BS}, G[i].visited^{BS}, G[i].isEdge^{BS})$
 $t_2^{BS} \leftarrow OSELECT(0, t_2, G[i].isEdge)$
end for

for $i = 1$ **to** N **do**
 $G[i].q^{BS} \leftarrow OSELECT(G[i].q^{BS}, 0, G[i].isEdge^{BS})$
end for

7.5 Complexity Analysis

- **Communication and Round Complexity:** The bottleneck is the bitonic sort, which requires $\mathcal{O}(\log^2(V + E))$ rounds. With two sorts per level

across $V - 1$ levels, the total round complexity is strictly bounded by $\mathcal{O}(V \cdot \log^2(V + E))$. The local computation and online bandwidth are exceptionally light and strictly linear, i.e., $\mathcal{O}(V + E)$ per phase, completely independent of the graph’s actual density.

- **Processing Overhead:** Without any optimizations, the algorithm has minimal processing overhead (standard MPC triples), however, with optimizations, the processing overhead can go up to that of permutation generation or even as massive as $\mathcal{O}(E)$ DPFs per level.
- **Security:** Topology is perfectly hidden. Access patterns depend strictly on public indices (sorting networks and linear increments 1 to N), preventing any degree or depth leakage.

7.6 Optional Optimizations to Eliminate Online Sorting

While the baseline OBFS protocol relies on Oblivious Bitonic Sort to dynamically route queue flags without pointer-chasing, the sorting network introduces a polylogarithmic $\mathcal{O}(\log^2 N)$ round and bandwidth overhead per iteration. Since the graph topology is static during a BFS traversal, we propose two theoretical optimizations to entirely eliminate sorting in the online phase.

7.6.1 Implicit Source Ordering and DUORAM Pointer-Chasing

The first optimization relies on structurally initializing the graph array and using PRAC’s underlying DUORAM for oblivious random access.

- **Eliminating Phase 1 sort:** In the preprocessing phase, the graph array G is initialized strictly in **source order**, i.e., each vertex is immediately followed by its outgoing edges. This static layout completely eliminates the need for the Phase 1 sort, allowing the source-to-edge propagation to occur via a single $\mathcal{O}(N)$ linear trickle-down scan.
- **Eliminating Phase 2 sort:** Instead of sorting incoming edges to sit alongside their destination vertices, each edge stores a direct DUORAM pointer to its destination vertex. The queue flag is passed by executing an oblivious memory update to the destination struct.
- **Complexity and Trade-offs:** To prevent topology leakage, a DUORAM update must be exe-

cuted for *every* edge E in the graph during every iteration, regardless of its active state. While PRAC’s DUORAM evaluates updates in $\mathcal{O}(1)$ online rounds, this approach requires generating $\mathcal{O}(|E|)$ Distributed Point Functions (DPFs) per level. Across $V - 1$ iterations, the preprocessing phase must generate $\mathcal{O}(V \cdot E)$ DPFs, exchanging the online round bottleneck for a massive offline bandwidth and storage overhead.

7.6.2 Pre-computed Permutations via Secure Shuffle

Taking inspiration from the Graphiti framework, we can exploit the static nature of the graph by replacing data-independent sorting with pre-computed permutations.

- **Mechanism:** In the offline phase, the parties compute the exact permutation $\pi_{S \rightarrow D}$ required to map the graph from **Source Order** to **Destination Order**, as well as the inverse permutation $\pi_{D \rightarrow S}$.
- **Online Execution:** During the online phase, the Oblivious Bitonic Sort is replaced by a constant-round Secure Shuffle. The parties apply the secret-shared permutation $\pi_{S \rightarrow D}$ to route flags from edges to destinations (Phase 2), and $\pi_{D \rightarrow S}$ to reset the array for the next level (Phase 1).
- **Complexity and Trade-offs:** A Secure Shuffle requires only $\mathcal{O}(1)$ rounds and strictly linear $\mathcal{O}(N)$ bandwidth. This strictly dominates both the baseline protocol and Optimization 1.

The different optimization strategies mainly show a trade-off between how much work is done online versus in preprocessing. Without any optimization, the protocol ends up being quite expensive during execution, taking $\mathcal{O}(\log^2 N)$ rounds and $\mathcal{O}(N \log^2 N)$ communication, but the preprocessing is minimal and straightforward. Optimization 1 flips this completely — it makes the online phase very efficient with just $\mathcal{O}(1)$ rounds and $\mathcal{O}(N)$ bandwidth, but at the cost of extremely heavy preprocessing involving $\mathcal{O}(E)$ DPFs at each level. Optimization 2 also achieves the same improvement in the online phase, but instead of such expensive preprocessing, it relies on generating permutations, which is much more manageable in practice. So while both optimizations improve performance significantly, Optimization 2 strikes a much better balance overall.

Table 1: Comparison of Complexities for Various Optimizations of PRAC-Based OBFS Protocol

Optimization	Online Rounds	Online Bandwidth	Preprocessing Overhead
No optimization	$\mathcal{O}(\log^2 N)$	$\mathcal{O}(N \log^2 N)$	Minimal (Standard MPC Triples)
Optimization 1	$\mathcal{O}(1)$	$\mathcal{O}(N)$	Massive ($\mathcal{O}(E)$ DPFs per level)
Optimization 2	$\mathcal{O}(1)$	$\mathcal{O}(N)$	Moderate (Permutation generation)

References

- [1] Kristján Valur Jónsson, Gunnar Kreitz, Misbah Uddin, *Secure Multi-Party Sorting and Applications*, 2011.
- [2] Amit Agarwal, Elette Boyle, Nishanth Chandran, Niv Gilboa, Divya Gupta, Yuval Ishai, Mahimna Kelkar, Yiping Ma, *Secure Sorting and Selection via Function Secret Sharing (FSS)*, 2024.
- [3] Gilad Asharov, Koki Hamada, Dai Ikarashi, Ryo Kikuchi, Ariel Nof, Benny Pinkas, Katsumi Takahashi, Junichi Tomida, *Efficient Secure 3-Party Sorting with Applications*, 2022.
- [4] Sajin Sasy, Adithya Vadapalli, Ian Goldberg, *PRAC: Round-Efficient 3-Party MPC for Dynamic Data Structures*, 2023.
- [5] Nishat Koti, Varsha Bhat Kukkala, Arpita Patra, Bhavish Raj Gopal, *Graphiti: Secure Graph Computation*, 2024.